Ernest Goh Chin Wee's Project Portfolio Page

Project: ScheduleFlow

Overview

ScheduleFlow is a Java 25 command-line application for university students managing coursework and fixed commitments. Students can record remaining work, generate a study plan within their available hours, and return to saved tasks, settings and schedules in later sessions.

Summary of Contributions

Given below are my contributions to the project.

- **New Feature: Automatic saving and loading**
  - **What it does:** FileStorage and StateCodec save and restore tasks, commitments, exact work/completion, study hours, policy, recurring templates, occurrence metadata and independent T/C/R counters in versioned UTF-8 text.
  - **Justification:** Students can resume work after closing the app without re-entering records or reusing deleted IDs.
  - **Highlights:** I implemented sibling temporary-file writing and atomic replacement, complete-file decoding, and an empty first-launch state only for a definitely absent file. Javadoc, runtime preconditions, internal assertions and opt-in `FINE` diagnostics document and protect the boundary. PR #24, PR #31. The team's shell and dispatcher orchestrate startup and commands.
- **New Feature: Restoring and validating frozen study plans**
  - **What it does:** PlanCodec stores a generated result and its full source in one replacement. Restart retains generation time, sessions, exact credited work, remainders and breaks without regenerating.
  - **Justification:** Students can return to the same schedule even after the current time changes.
  - **Highlights:** I extended storage and startup/dispatch integration, preserved supported legacy formats, and reconciled recurrence metadata. Successful source changes invalidate stale plans; failed saves retain the previous state and plan. PR #89 addresses issue #68. The collaborative release follow-up now connects Gabriel's `Planner.validate(Plan)` to encoding and decoding. Invalid intervals or work accounting are rejected before restoration or filesystem changes; valid frozen schedules remain unchanged. Gabriel owns the planning rules; this integration reuses his boundary.
- **Code contributed:** RepoSense code view | GitHub profile | HTML portfolio. My main contribution was persistence through `FileStorage`, `StateCodec` and `PlanCodec`. I also contributed task-command parsing, save/dispatch integration and study-policy persistence, building on the team's task services, allocator and rendering. Updated 1 November 2026 | Source: master 1903276
- **Team contributions:**
  - **Integration:** Reconciled persistence with teammates' exact-work, recurrence, hours and policy changes, retaining their metadata and saving source/plan changes together. PR #84, PR #89, PR #90.
  - **Authorship allocation:** Applied the agreed whole-file RepoSense allocation to storage, `Snapshot` and assigned tests (PR #94). Tags record maintenance responsibilities; Git history retains original implementation credit.

- **Enhancements to existing features:**
  - **Storage failure hardening:** I added a package-private filesystem seam and real-file regressions for creation failure, partial writes, unsupported atomic moves, replacement errors and failed cleanup (PR #124). Tests check retained target bytes/plans, cleanup, original causes and suppressed cleanup errors. Normal operations still use `Files` with no non-atomic fallback. Injected faults are not proof of universal provider support or power-loss durability.
  - **Progress and remaining estimates:** Added parsing for both progress forms, connected validated candidates to saving/publication, and completed remaining-time revision. Covered no-ops, invalid inputs, failed saves and restarts. This uses Nickhil's lifecycle services; Ka How completed progress confirmation rendering. PR #77, PR #84, PR #83.
  - **Completion and policy controls:** Connected completion and completed-task viewing to the console. Added policy set/show/reset, persisted settings, and supplied them to Gabriel's continuous-work allocator. Equal settings preserve the plan; changes save before publication. PR #90.
  - **Multi-date parsing:** Added continuous/daily unavailable-period parsing and malformed-input tests in PR #88. Darren subsequently completed the dispatcher routes and atomic batches in PR #126; the commands now work. My contribution was parsing, not his dispatch work.
  - **Regression coverage:** Added JUnit 5 codec/storage coverage for Unicode and pipe-containing names, legacy/current formats, deleted-ID counters, exact work, corrupt input and frozen-plan round trips. FileStorageTest uses temporary folders and fresh readers; StateCodecTest and PlanCodecTest isolate text conversion. Current release regressions reject infeasible plans, preserve input bytes/validation causes and accept alternative valid frozen schedules.
  - **Portability and integration tests:** I also extended parser, dispatcher and application tests for saved restarts, no-op plan preservation and failed-save rollback, using fixed clocks where needed. Corrected a startup process test to compare filesystem identity across macOS aliases and Windows short names while retaining exact error framing and data-preservation checks (PR #109).
- **Documentation:**
  - **User Guide:** Wrote/updated the quick start, duration guidance, task listing, frozen-plan example, saving/restoration and recovery FAQs. Clarified exact work versus reserved capacity, data location, backups, compatibility and plan invalidation.
  - **Developer Guide:** Wrote the assigned persistence design, atomic-save/restoration explanation, storage verification, NFRs and manual recovery tests, plus acknowledgements. Created the original storage class and atomic-save sequence diagrams (PR #107); updated my sources/renders for the failure seam in PR #124. Subsequent team revisions refine UML notation and show validation.
- **Community:**
  - Reviewed rendering (PR #22, PR #87), planner-policy compatibility (PR #81 review), console documentation (PR #100), UML/source consistency (PR #120 review), output-failure handling and CI (PR #123 review), and release validation/documentation scope (PR #134 review). My output-failure review checked that durable saves are retained after failed confirmations.